

Aperture — Implementation Playbook

Version: v1.0

Document Type: Engineering Playbook

Related Documents

- Development Roadmap
 - System Architecture
 - Technical Architecture
 - Backend Architecture
 - Frontend Architecture
 - Database Design
-

1. Purpose

The Implementation Playbook defines the recommended order for building Aperture.

The objective is to:

- Keep the application working after every milestone
- Reduce architectural rework
- Build dependencies before dependents
- Produce meaningful Git commits
- Reach a deployable MVP quickly
- Layer advanced features incrementally

Every phase should end with a working, testable application.

2. Guiding Principles

Implementation should follow these rules:

- Build infrastructure before features
- Build foundations before optimization
- Complete one vertical slice at a time
- Avoid speculative abstractions
- Maintain deployable software throughout development

Each phase should add value without requiring unfinished future work.

3. Definition of Done

A feature is complete only when it includes:

- Implementation
- Validation
- Error handling
- Tests
- Documentation updates
- Logging
- Basic performance review

Code alone does not constitute completion.

4. Phase 0 — Repository Foundation

Objectives

Create a production-quality development environment.

Deliverables

Repository initialization

Backend project

Frontend project

Docker Compose

Environment configuration

Pre-commit hooks

Formatting

Linting

GitHub repository

README

LICENSE

Infrastructure

- FastAPI
 - Next.js
 - PostgreSQL
 - Docker
 - GitHub Actions (basic)
-

Exit Criteria

Developers can clone the repository and run the application with:

```
docker compose up
```

The frontend and backend communicate successfully.

5. Phase 1 — Authentication & Identity

Objectives

Establish user identity.

Features

- Registration
 - Login
 - JWT authentication
 - Refresh tokens
 - Google OAuth
 - User profile creation
-

Technical Goals

Authentication middleware

Protected routes

User context

Session management

Exit Criteria

Users can register, authenticate, and access protected endpoints.

6. Phase 2 — Metadata Platform

Objectives

Build the content foundation.

Features

TMDb integration

Movie import

TV show import

People

Genres

Collections

Images

Movie detail pages

TV detail pages

Technical Goals

Metadata synchronization

Caching

Canonical schema

Normalization

Exit Criteria

Users can browse and search imported content.

7. Phase 3 — Community Foundation

Objectives

Introduce user-generated content.

Features

Ratings

Reviews

User profiles

Follow system

Basic activity

Technical Goals

Review service

Relationship modeling

Pagination

Validation

Exit Criteria

Users can review content and interact with profiles.

8. Phase 4 — Lists & Curation

Objectives

Expand community engagement.

Features

Watchlists

Custom lists

Favorites

Public lists

List ordering

Technical Goals

List APIs

Ordering logic

Permissions

Caching

Exit Criteria

Users can curate and share collections.

9. Phase 5 — Discovery Platform

Objectives

Transform Aperture into a discovery product.

Features

Homepage

Trending

Editorial collections

Recommendations

Similar titles

Continue browsing

Technical Goals

Feed orchestration

Ranking

Aggregation

Parallel data loading

Exit Criteria

Homepage becomes dynamic and personalized.

10. Phase 6 — Search Platform

Objectives

Improve discoverability.

Features

Full-text search

Autocomplete

Filters

Sorting

Search history (optional)

Technical Goals

PostgreSQL search

Query optimization

Caching

Search abstraction

Exit Criteria

Users can efficiently locate movies, TV shows, people, and lists.

11. Phase 7 — Streaming Platform

Objectives

Answer "Where can I watch this?"

Features

Streaming providers

Regional availability

Provider filtering

Watch options

Technical Goals

Synchronization pipeline

Region resolution

Cache invalidation

Exit Criteria

Every movie and TV page displays current streaming availability.

12. Phase 8 — Polish & User Experience

Objectives

Elevate product quality.

Features

Animations

Skeleton loaders

Accessibility improvements

Responsive refinements

Dark/light themes

Error states

Empty states

Technical Goals

Performance optimization

Code splitting

Image optimization

Accessibility review

Exit Criteria

The application feels production-ready across devices.

13. Phase 9 — AI & Personalization

Objectives

Differentiate Aperture.

Features

Hybrid recommendations

Semantic similarity

Cinematic DNA

Natural language discovery (future)

Explainable recommendations

Technical Goals

Embeddings

pgvector

Recommendation pipelines

Background processing

Exit Criteria

Recommendations extend beyond metadata similarity and reflect user behavior.

14. Phase 10 — Production Readiness

Objectives

Prepare for public deployment.

Features

Monitoring

Logging

Health checks

Rate limiting

Security review

Backups

Operational dashboards

Technical Goals

CI/CD

Infrastructure hardening

Observability

Performance tuning

Exit Criteria

Aperture is suitable for continuous public deployment.

15. Parallel Workstreams

Some work can proceed concurrently.

Frontend

- Components
 - Design system
 - Layouts
 - Responsive behavior
-

Backend

- APIs
 - Services
 - Database
-

DevOps

- CI/CD
 - Docker
 - Infrastructure
-

Documentation

- ADRs
- API updates
- Architecture refinements

Independent workstreams reduce bottlenecks.

16. Milestone Reviews

At the end of every phase, perform a review covering:

- Functional completeness
- Test coverage
- Performance
- Security
- Documentation
- Technical debt

Do not advance while major architectural issues remain unresolved.

17. Git Strategy

Recommended branching model:

- `main`
- `develop`
- Short-lived feature branches

Each pull request should:

- Pass CI
- Include tests
- Update documentation when necessary
- Receive review before merging

Commits should remain focused and descriptive.

18. Deployment Strategy

Deployment evolves with the project.

Development

↓

Local Docker

↓

Preview Deployments

↓

Staging

↓

Production

Every environment should be reproducible from version-controlled configuration.

19. Success Metrics

The project is successful when it demonstrates:

Product

- Rich discovery experience
 - Community engagement
 - Useful recommendations
-

Engineering

- Clean architecture
 - Strong documentation
 - Automated testing
 - Reliable deployments
 - Observability
-

Portfolio

An interviewer should be able to understand:

- Why each architectural decision was made
 - How the system evolves
 - How the project would scale
 - How engineering tradeoffs were evaluated
-

20. Summary

The Implementation Playbook converts Aperture's architecture into a practical engineering roadmap.

Rather than attempting to build every feature simultaneously, it emphasizes incremental delivery, stable foundations, and production-quality engineering practices from the beginning.

By completing each phase before moving to the next, the project remains deployable, maintainable, and continuously demonstrable throughout development.